

Python for HPC

SDSC Summer Institute 2026

Andrea Zonca

Friday, August 7 | 8:30 AM to 11:20 AM

**SAN DIEGO
SUPERCOMPUTER
CENTER**

UC San Diego

HALICIOĞLU SCHOOL OF DATA SCIENCE
AND COMPUTING

What you will be able to do

- Compile and benchmark a numerical loop with Numba.
- Choose threads or processes from workload evidence.
- Choose Dask chunks and scale one expression across nodes.

The slides lead the session

- Follow the file or action shown here.
- Work in the notebook while this slide stays visible.
- Return here for the debrief and next step.
- Blue means stuck. Yellow means ready.

Today has one core path

- Setup and performance workflow.
- Numba, then threads and processes.
- Dask on one node, then two nodes.
- AI review, recap, and take-home material.

1 of 8 | Setup

- Time check: 8:30 AM.
- Goal: open Jupyter and verify the environment.
- We move on when most laptops show yellow.

SETUP | Open the lesson

- In the repository, open folder 6.1_python_for_HPC.
- Run `bash launch_galileo_compute.sh`.
- Keep one Jupyter terminal available.

SETUP | Run the import check

- In Jupyter, open the lesson README.md.
- Copy and run the four-package import cell.
- Yellow means imports work.
- Blue means a helper should stop by.

SETUP | Start with evidence

- Verify the answer.
- Time or profile a baseline.
- Change one layer at a time.
- Verify again and compare steady-state cost.

2 of 8 | Numba

- Time check: 8:42 AM.
- Goal: compile one hot loop and benchmark fairly.
- Core file: `3\_numba/0\_basics.ipynb`.

NUMBA | jit and njit today

- Since Numba 0.59, bare @jit uses nopython mode.
- @njit is the explicit alias for @jit(nopython=True).
- This lesson uses @njit to make the intent visible.
- The first call compiles an input signature.

NUMBA | Open this notebook

- Open folder 3_numba.
- Open 0_basics.ipynb.
- Run through "Compare with vectorized NumPy."
- Predict each timing before you run it.
- Stop at "Your turn."

NUMBA | Pause and ask

- Did all three versions return the same value?
- Which timing includes compilation?
- Why do we warm up before benchmarking?

NUMBA | Hands-on

- Complete sum_of_squares.
- Check the result against NumPy.
- Warm up, then time both versions.
- 12 minutes. Blue means help. Yellow means ready.

NUMBA | Debrief

- First call: compilation plus execution.
- Later calls: steady-state execution.
- @njit helps supported numerical loops.
- Time check: 9:15 AM.

3 of 8 | Threads and processes

- Time check: 9:15 AM.
- Goal: predict the useful execution model.
- Notebook: threads versus processes.

SCHEDULERS | Mental model

- Threads share memory and one interpreter.
- The GIL limits simultaneous pure-Python bytecode.
- Processes use separate interpreters and memory.
- Startup and serialization have a cost.

SCHEDULERS | Open this notebook

- Open folder 4_threads_vs_processes.
- Open threads_vs_processes.ipynb.
- Read the mental model.
- Predict both comparisons before running them.
- Stop at "Your turn."

SCHEDULERS | Pause and ask

- Which workload is CPU-bound pure Python?
- Which workload mostly waits?
- What data would be expensive to send to a process?

SCHEDULERS | Hands-on

- Run both schedulers with four workers.
- Explain each result with the mental model.
- Discuss one surprise with a neighbor.
- 12 minutes. Blue means help. Yellow means ready.

SCHEDULERS | Debrief

- CPU-bound Python often favors processes.
- Waiting work often favors threads.
- Shared data can change the tradeoff.
- Time check: 9:40 AM.

4 of 8 | Dask tasks and chunks

- Time check: 9:40 AM.
- Goal: build a graph, then choose useful chunks.
- Notebooks: delayed, then multicore array.

DASK | Graph before execution

- Delayed calls build tasks.
- Dependencies form a directed acyclic graph.
- The scheduler decides when and where tasks run.
- `compute()` triggers execution.

DASK | Open the delayed notebook

- Open folder 5_dask.
- Open 1_delayed.ipynb.
- Run through "Delay calls, then compute once."
- Inspect what exists before compute().
- Stop at "Your turn."

DASK | Delayed exercise and debrief

- Put the reduction inside the graph.
- Compute only the final summary.
- 5 minutes. Blue means help. Yellow means ready.
- Then explain what `compute()` changed.

DASK | Chunks are units of work

- Too small: scheduling overhead dominates.
- Too large: memory pressure and weak parallelism.
- Useful chunks fit memory and keep workers busy.
- Inspect chunks before `compute()`.

DASK | Open the array notebook

- In folder 5_dask, open 2_multicore_array.ipynb.
- Run through "Choose chunks."
- Compare the eager and lazy objects.
- Stop at "Your turn."

DASK | Chunk exercise

- Try chunk sides 250, 1000, and 3000.
- Keep the mathematical expression unchanged.
- Compare task count, memory, and timing.
- 10 minutes. Blue means help. Yellow means ready.

DASK | Debrief

- A graph is a plan, not a result.
- Chunks control scheduling and memory.
- NumPy can still win for small in-memory work.
- Time check: 10:10 AM.

5 of 8 | Break and cluster setup

- Time check: 10:10 AM.
- Break ends at 10:20 AM.
- Helpers can start cluster setup with ready learners.

CLUSTER SETUP | Two terminals

- In a Jupyter terminal, open folder `dask_slurm`.
- Run `bash launch_scheduler.sh`.
- In a login terminal, run `sbatch dask_workers.slm`.
- Wait until the scheduler reports two workers.

6 of 8 | Multi-node capstone

- Time check: 10:20 AM.
- Goal: run one Dask expression on two worker nodes.
- Notebook: multi-node distributed array.

CAPSTONE | Three roles

- Notebook builds the graph and requests a result.
- Scheduler tracks dependencies and assigns tasks.
- Workers hold chunks and execute tasks.
- The array expression does not change.

CAPSTONE | Open this notebook

- Open folder 5_dask.
- Open 4_multinode_distributed_array.ipynb.
- Run "Start the cluster."
- Confirm two distinct worker hosts.
- Stop before "Build a distributed array."

CAPSTONE | Run and verify

- Build and compute the distributed array.
- Read worker hosts, threads, and result.
- Confirm the expected value.
- 18 minutes. Blue means help. Yellow means ready.

CAPSTONE | Debrief and clean up

- What changed when we added nodes?
- What stayed the same in the Python expression?
- Cancel the worker job and stop the scheduler.
- Time check: 10:52 AM.

7 of 8 | AI-assisted workflow

- Time check: 10:52 AM.
- Goal: turn a suggestion into a tested hypothesis.
- Page: AI code-assist README.

AI | A reviewable loop

- State the environment, limits, and correct result.
- Ask for one small change and an explanation.
- Inspect commands and resource requests.
- Test correctness, then performance.

AI | Open this page

- Open folder 2_ai_code_assist.
- Open README.md.
- Read "A useful HPC prompt."
- Choose one function from the Numba notebook.
- Stop at "Hands-on: ask, inspect, test."

AI | Ask, inspect, test

- Ask for one Numba optimization.
- Identify every package and resource assumption.
- Run the correctness check and benchmark twice.
- 8 minutes. Accept, revise, or reject.

AI | Debrief

- Plausible output is not evidence.
- Generated SLURM needs a line-by-line resource review.
- Keep only a verified improvement.
- Time check: 11:08 AM.

8 of 8 | Recap

- Time check: 11:08 AM.
- Goal: choose the smallest useful layer.
- Final questions end at 11:20 AM.

RECAP | Decision map

- Hot numerical loop: Numba.
- Waiting tasks: threads or delayed.
- CPU-bound Python: processes.
- Chunked or multi-node array: Dask.

RECAP | What you take home

- Tested core and optional notebooks.
- Production SI26 SLURM and Galyleo templates.
- A debug-queue validation workflow.
- Time check: 11:20 AM. Questions?